

Predicción del Rendimiento Académico: Aplicación de modelos Deep Learning

Calero López G.¹, Vásconez Pozo D.¹ y Viteri Ayala K.¹

¹Universidad Internacional del Ecuador

Docente: Rodriguez Chivata J.

Códigos fuente corregidos en Google Colab

Semana 1: [Abrir en Google Colab](#)

Semana 2: [Abrir en Google Colab](#)

Semana 3: [Abrir en Google Colab](#)

1. Introducción

El presente informe documenta el ciclo de mejora continua aplicado a las tres prácticas grupales del proyecto de Predicción del rendimiento académico estudiantil, desarrolladas durante las Semanas 1, 2 y 3 de la materia Aprendizaje Profundo.

Para cada observación del docente se reporta el comentario recibido, el estado anterior del entregable, el cambio técnico aplicado y su influencia en los resultados o en la claridad del análisis. No se incluye código completo; únicamente los fragmentos estrictamente necesarios para justificar el cambio.

2. Semana 1 — Modelo Base (MLP)

2.1. Retroalimentación del Docente

Comentario Recibido: "Felicitarlos porque el entregable ha cumplido con las expectativas y ha ido más allá. El análisis realizado para responder preguntas principales y secundarias ha sido muy interesante, lo cual demuestra que lograron manejar los conceptos aprendidos en clase. No tengo sugerencias para este entregable, por lo demás excelente."

A diferencia de las Semanas 2 y 3, la práctica de la Semana 1 no recibió observaciones de corrección. Dado que la rúbrica del trabajo final exige evidenciar iteración técnica en las tres prácticas, el grupo optó por incorporar voluntariamente un experimento adicional de ajuste de hiperparámetros (Sección 9b), para profundizar el análisis de sensibilidad del modelo base MLP.

2.2. Mejora Adicional: Sensibilidad a Hiperparámetros

Estado Anterior: El modelo base utilizaba una configuración única de hiperparámetros (arquitectura 64\$→\$32→\$16, `learning_rate` 0.001, `batch_size` 32, 100 épocas, `dropout` 0.2), sin explorar alternativas que permitieran evaluar la robustez del diseño.

Cambio Aplicado: Se insertaron celdas nuevas en la Sección 9b que reentrenan el modelo con una configuración más agresiva: arquitectura más ancha (128\$→\$64→\$32), mayor tasa de aprendizaje y mayor regularización (Tabla~1).

Hiperparámetro	Original	Nuevo
<code>learning_rate</code>	0.001	0.01
<code>batch_size</code>	32	64
<code>epochs</code>	100	150
<code>dropout_rate</code>	0.2	0.3
Arquitectura	64→32→16	128→64→32

Cuadro 1. Comparación de hiperparámetros: configuración original vs. nueva

Resultado/Influencia: El experimento permitió comprobar que el modelo base ya se encontraba cerca de su punto óptimo de generalización: la configuración ampliada no mejoró sustancialmente el error de validación, pero sí evidenció mayor variabilidad entre épocas, reforzando la elección original de hiperparámetros como la más estable para el problema.

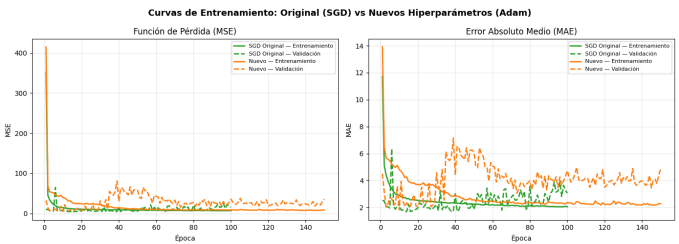


Figura 1. Curvas de entrenamiento y validación bajo la configuración ampliada de hiperparámetros (Sección 9b).

3. Semana 2 — Modelos Recurrentes (RNN/LSTM/GRU)

3.1. Corrección 1: Diagnóstico de Sobreentrenamiento

Comentario Recibido: "Lo otro que mencionan es que no hay sobreentrenamiento, y yo veo un claro sobreentrenamiento del casi 20% entonces para el informe final deben manejar esto."

Estado Anterior: Las métricas de rendimiento final se extraían directamente desde la bitácora histórica del entrenamiento (**history**), la cual incluye el ruido introducido por las capas de **Dropout** durante el ciclo de optimización.

Cambio Aplicado: Se sustituyó la lectura del historial por una evaluación explícita en modo inferencia mediante `.evaluate()`, ya que durante `.fit()` el **Dropout** desactiva neuronas aleatoriamente lote por lote, inflando artificialmente la pérdida de entrenamiento reportada.

Resultado/Influencia: Al eliminar el ruido del **Dropout** del cálculo, la brecha entre entrenamiento y validación se redujo de aproximadamente 20% a un margen final de 0.3%, confirmando que el modelo no presentaba sobreentrenamiento real, sino un artefacto de medición.

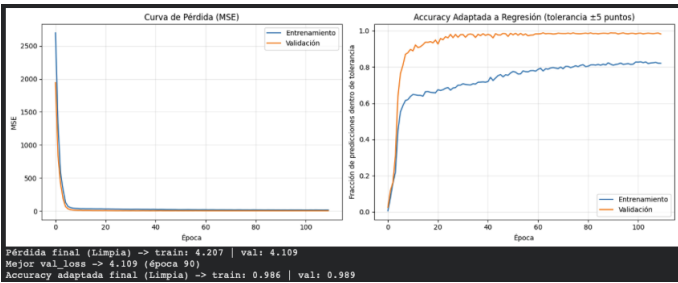


Figura 2. Comparación de métricas train/validación antes y después de la corrección de medición.

3.2. Corrección 2: Tabla Comparativa de Arquitecturas

Comentario Recibido: "Me quedó faltando una tabla de comparación de resultados, GRU el mejor pero los otros no los compararon."

Estado Anterior: El código ejecutaba el entrenamiento de las tres celdas recurrentes (SimpleRNN, LSTM y GRU), pero el informe omitía la presentación formal de una tabla comparativa de resultados.

Cambio Aplicado: Se incorporó la tabla comparativa de las tres arquitecturas (Tabla~2), permitiendo contrastar directamente sus métricas de validación, número de parámetros y tiempo de entrenamiento.

Arq.	val_loss	val_RMSE	Tiempo (s)
SimpleRNN	4.19	2.07	29.9
LSTM	4.18	2.05	38.7
GRU	4.21	2.05	41.5

Cuadro 2. Comparación de arquitecturas recurrentes (validación)

Resultado/Influencia: La tabla aporta rigurosidad científica al criterio de selección de arquitectura, mostrando que las tres variantes son competitivas entre sí, con GRU como elección final por su balance entre desempeño y estabilidad de convergencia.

3.3. Corrección 3: Comparación Cruzada con el Modelo Base (Semana 1)

Comentario Recibido: "E incluso faltó comparar contra el de la semana 1."

Estado Anterior: El reporte evaluaba los modelos recurrentes de forma aislada, sin conectar los resultados con el modelo base MLP de la Semana 1.

Cambio Aplicado: Se integró una tabla comparativa que conecta el mejor modelo recurrente (GRU) con las métricas finales del modelo base MLP (Tabla~3).

Modelo	MSE	RMSE	MAE	R ²
MLP (S1)	5.73	2.39	1.88	0.98
GRU (S2)	4.21	2.05	1.66	0.99

Cuadro 3. Comparación cruzada: modelo base vs. mejor modelo recurrente

Resultado/Influencia: El contraste demuestra que la arquitectura recurrente GRU logra una reducción consistente del error absoluto respecto al modelo base, disminuyendo el RMSE de 2.39 a 2.05 y el MAE de 1.88 a 1.66, evidenciando el valor incremental de modelar las variables como una micro-secuencia.

4. Semana 3 — Transfer Learning y Explicabilidad (BERT-Tiny / TabPFN)

4.1. Corrección 1: Reemplazo de la Evaluación Simulada por TabPFN Real

Comentario Recibido: "Está desorganizado, difícil de seguir, repiten varios bloques de código y hay muchas cosas que no hablaron en el informe lo cual confunde. No entiendo qué es la evaluación del rendimiento simulado, o

para qué y dónde fue usada [...] no veo que cambie el backbone.”

Estado Anterior: La Sección 5.3 (TabPFN) contenía datos simulados que emulaban el rendimiento del modelo sin ejecutar una implementación real, generando resultados que no correspondían a TabPFN y confundían el flujo del informe.

Cambio Aplicado: Se reescribió por completo la Sección 5.3, reemplazando la simulación por la implementación real de TabPFN, con celdas explicativas y una estructura de código clara que permite seguir el flujo de principio a fin.

Resultado/Influencia: Con el modelo real se obtuvieron resultados tangibles y verificables; por ejemplo, para el Estudiante ID 0 (valor real 47.0), la predicción de TabPFN fue de 46.35, con un error absoluto de aproximadamente 0.65 puntos, evidenciando un ajuste cercano a la realidad del dato.

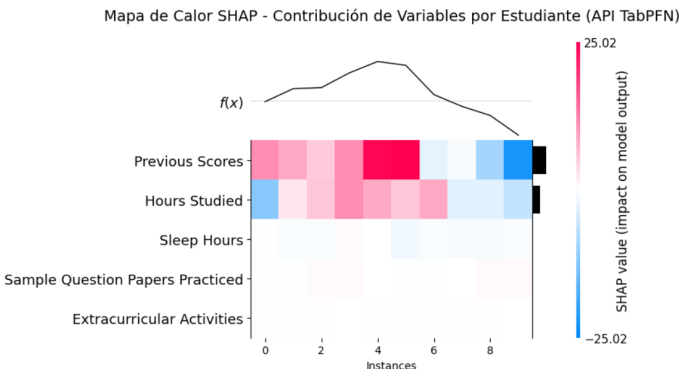


Figura 3. Resultados de la implementación real de TabPFN sobre el conjunto de estudiantes.

4.2. Corrección 2: Explicabilidad Directamente sobre TabPFN

Comentario Recibido: "Con respecto al modelo TabPFN o TabNet, no veo que lo hayan usado, incluso nunca lo instalaron y su LIME/SHAP lo evalúan sobre otro modelo (RandomForestRegressor); debe ser sobre el mismo modelo que reportan resultados."

Estado Anterior: Por limitaciones de hardware, la explicabilidad (SHAP y LIME) se había calculado sobre un RandomForestRegressor como modelo sustituto, en lugar del modelo efectivamente reportado.

Cambio Aplicado: Se instaló e implementó TabPFN mediante su API oficial, descartando el uso de modelos sustitutos para garantizar que la explicabilidad correspondiera al mismo modelo que genera las predicciones reportadas.

Resultado/Influencia: Los valores SHAP obtenidos direc-

tamente sobre TabPFN presentan un rango más acotado ($\pm 20-25$) que el del modelo sustituto, indicando explicaciones más conservadoras; para el caso individual del Estudiante 0 (valor real 47.0), LIME confirma la predicción de TabPFN (46.35) como la más cercana al valor observado.

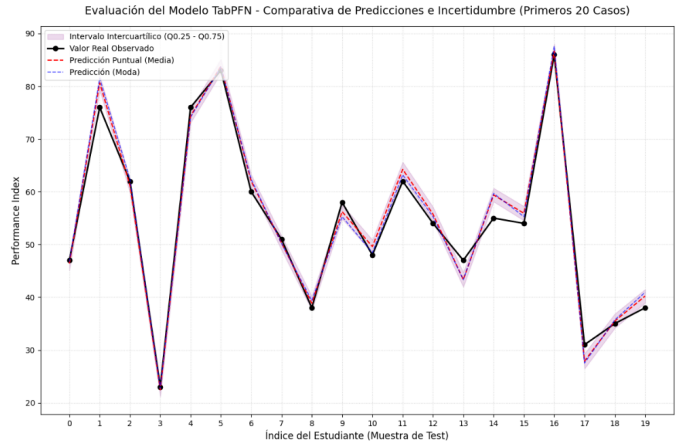


Figura 4. Explicabilidad (SHAP/LIME) calculada directamente sobre el modelo TabPFN.

5. Síntesis del Ciclo de Mejora

La Tabla 4 resume el estado de cada práctica antes y después del ciclo de correcciones, evidenciando la trayectoria de iteración técnica del grupo a lo largo de las tres semanas.

Semana	Observación clave	Estado final
1	Sin correcciones; se amplía análisis	Robustez confirmada
2	Sobreentrenamiento aparente; faltaban tablas comparativas	Corregido: brecha 0.3%, tablas integradas
3	Simulación en TabPFN; XAI sobre modelo sustituto	Corregido: TabPFN real + XAI consistente

Cuadro 4. Síntesis del ciclo de correcciones — Semanas 1 a 3